Imagine you've just deployed a new app to production when a customer calls saying that they're getting an error message using your app. How would you identify what caused the problem? You could ask the customer what steps they were taking and potentially try to re-create the error yourself, but if that doesn't work, you're left trawling through the code, trying to spot errors with nothing else to go on.

Logging can provide the extra context you need to quickly diagnose a problem. Arguably, the most important logs capture the details about the error itself, but the events that led to the error can be equally useful in diagnosing the cause of an error.

There are many reasons for adding logging to an application, but typically, the reasons fall into one of three categories:

- Logging for auditing or analytics reasons, to trace when events have occurred
- Logging errors
- Logging nonerror events to provide a breadcrumb trail of events when an error does occur

The first of these reasons is simple. You may be required to keep a record of every time a user logs in, for example, or you may want to keep track of how many times a particular API method is called. Logging is an easy way to record the behavior of your app by writing a message to the log every time an interesting event occurs.

I find the second and third reasons for logging to be the most common. When an app is working perfectly, logs often go completely untouched. It's when there's a problem and a customer comes calling that logs become invaluable. A good set of logs can help you understand the conditions in